2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2025)
October 19-25, 2025. Hangzhou, China

2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) | 979-8-3315-4393-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/IROS60139.2025.11246251

Experimental Evaluation of Safe Trajectory Planning for an Omnidirectional UAV

Mahmoud Hamandi[1], Abdullah Mohamed Ali[1], Anthony Tzes[1], Farshad Khorrami[2]

***Abstract*— Autonomous aerial vehicles play a critical role in search and rescue operations, where navigation through cluttered and confined environments is essential. To this end, this paper presents a novel trajectory planning framework for omnidirectional drones that dynamically adjusts tracking velocity based on the platform's proximity to obstacles, ensuring a balance between safety and efficiency in cluttered and challenging environments. The proposed approach generates a geometric path to the target location. At each waypoint, the minimum distance between the drone's convex hull and surrounding obstacles is determined, allowing the computation of the velocity constraints. By slowing down near obstacles and accelerating in open spaces, the method enhances both safety and maneuverability. The framework is validated through real-world experiments using the OmniOcta UAV, demonstrating its ability to navigate through constrained spaces. Furthermore, we present an experimental study to investigate key sources of tracking deviations, including propeller dynamics and aerodynamic interactions near obstacles.**

I. INTRODUCTION

Search and rescue scenarios pose significant challenges due to the unpredictable and hazardous conditions of disaster zones, making human intervention difficult. Consequently, robotic systems capable of navigating these harsh environments, scanning for survivors and viable paths through debris, and retrieving critical information and equipment are being developed by researchers and industry [1]. Other applications also require UAVs for the inspection of complex 3D spaces, particularly in environments with dense obstacles, such as industrial facilities, underground tunnels, confined infrastructure, and others [2]–[4].

In this perspective, Unmanned Aerial Vehicles (UAVs) are advantageous over other robotic systems due to their workspace spanning the entire three-dimensional aerial space [5]. Despite this advantage, classical coplanar/collinear UAVs have a restricted motion due to the coupling between their lateral and angular motions. As such, it is impossible for such platforms to navigate challenging environments requiring traversal at acute angles, such as vertical windows. Intuitively, such spaces could be traversed by increasingly smaller UAVs that could navigate small openings. However, such a solution limits the size of the UAV and the corresponding payload, which might be crucial for the discovery of the environment.

In this perspective, many researchers have focused on the design of Fully Actuated UAVs [6] with decoupled lateral and angular motions. Omnidirectional designs [7] further expand the workspace of fully actuated UAVs allowing hovering in any orientation; thus, providing a more agile and maneuverable robotic system due to its ability to navigate challenging environments.

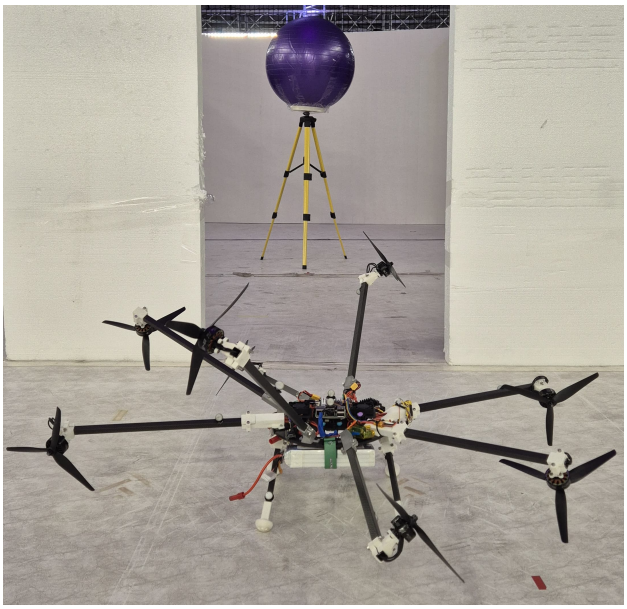


Fig. 1: Omnidirectional drone in front of an obstacle track, with a tight vertical opening followed by a sphere.

This paper focuses on trajectory generation for omnidirectional UAVs operating in cluttered and challenging environments, such as navigating through narrow vertical openings, as illustrated in Figure 1. The proposed method was validated in real-world conditions using a prototype omnidirectional UAV with fixed unidirectional thrusters introduced in our previous work [8], [9].

Traditional trajectory planning methods for quadrotors have predominantly relied on optimization techniques and polynomial trajectory generation to ensure obstacle avoidance. For instance, Gao et al. [10] present a gradient-based optimization framework that generates smooth and dynamically feasible paths while ensuring collision-free navigation. Similarly, Mellinger and Kumar [11] employ polynomial trajectory generation to achieve real-time motion planning that could allow quadrotors to navigate cluttered environments. While such approaches have been widely adopted for underactuated quadrotors, they do not fully leverage the capabilities of omnidirectional UAVs. Exploiting the full actuation of such platforms necessitates a trajectory planning framework that accounts for their increased maneuverability, allowing for efficient navigation in constrained spaces without relying on aggressive or high-speed maneuvers.

Recent works have demonstrated that conventional quadro-

[1] Center for Artificial Intelligence and Robotics, New York University Abu Dhabi, Saadiyat Island, 129188, Abu Dhabi, UAE abdullah.ali@nyu.edu, mahmoud.hamandi@nyu.edu, anthony.tzes@nyu.edu

[2] New York University, Brooklyn, NY 11201, USA khrorrami@nyu.edu

This work has been supported by the NYUAD Center for Artificial Intelligence and Robotics, funded by Tamkeen under the NYUAD Research Institute Award CG010.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 04:05:14 UTC from IEEE Xplore. Restrictions apply.

tors can execute challenging maneuvers such as passing through narrow gaps [12], [13]. However, these approaches typically involve executing the maneuver at high velocity, often requiring the UAV to enter free fall upon exit. While effective, this aggressive strategy results in increased tracking errors due to the inherent instability of such maneuvers, rendering ineffective in scenarios where precision is critical.

Trajectory planning for fully actuated multirotor systems has been explored in the literature, with a particular emphasis on computational efficiency and energy optimization. Brescianini et al. [14] proposed a trajectory computation strategy designed for rapid computation while ensuring dynamic feasibility of the computed trajectory. However, their approach does not explicitly account for environmental constraints or obstacles, making it less suitable for cluttered environments. Likewise, Morbidi et al. [15] introduce an optimization framework to minimize energy consumption for a fully actuated hexarotor. While effective in reducing power usage, this method lacks an integrated obstacle avoidance mechanism, limiting its practicality in some navigation scenarios. Pantic et al. [16] introduced a motion planning approach based on Riemannian geometry for six-degree-of-freedom (6-DoF) trajectory generation. Their method ensures smooth and dynamically feasible paths but, similar to previous works, does not explicitly address obstacle avoidance, a crucial requirement for UAV operation in cluttered environments.

Liu et al. [17] proposed a framework for generating collision-free 6D trajectories in cluttered environments. Their method models traversable free space as a collection of convex polyhedra and computes safe trajectories that ensure the UAV's bounding box remains within these regions. While promising, their approach assumes fixed velocity limits, inherently favoring high-speed trajectory generation. However, increased speed can lead to significant tracking errors, necessitating a reduction in velocity when maneuvering near obstacles to maintain accurate path tracking. Furthermore, the bounding box approximation simplifies computation but may exclude viable solutions that could be feasible with a more precise geometric representation of the UAV during the planning stage.

To fully take into account the maneuverability of omnidirectional UAVs, trajectory generation frameworks must be developed that integrate both obstacle-aware path planning and the unique actuation capabilities of omnidirectional UAVs, ensuring safe and efficient navigation in constrained environments. Unlike conventional UAVs, omnidirectional UAVs have full six degrees of freedom (6-DoF), allowing both position and orientation adjustments during flight. This enables path planning to account for not only spatial navigation but also the necessary rotational adaptations required to traverse constrained environments where both translation and reorientation are essential for safe passage.

This work extends [18] with the following contributions:

- Propose a multi-stage trajectory generation algorithm for the navigation of omnidirectional UAVs in cluttered and challenging environments. The proposed method will balance between safety near obstacles and minimum travel time in open spaces.
- Experimentally validate the proposed method in a cluttered environment with the omnidirectional platform presented in our previous work [8].
- Experimentally assess sources of trajectory tracking errors near obstacles for the omnidirectional drone, and propose solutions for safe navigation near obstacles.

II. Mathematical Modeling

A. Dynamical Model

A fixed world frame $\mathcal{F}_W$ is assumed within the environment with corresponding origin and axes $\{O_W, \boldsymbol{x}_W, \boldsymbol{y}_W, \boldsymbol{z}_W\}$. A fixed frame $\mathcal{F}_B$ is attached to the Center of Mass (CoM) of the OmniOcta, with corresponding origin and axes $\{O_B, \boldsymbol{x}_B, \boldsymbol{y}_B, \boldsymbol{z}_B\}$. The position (orientation) of $\mathcal{F}_B$ in $\mathcal{F}_W$ (with respect to $\mathcal{F}_W$) is denoted as ${}^W\boldsymbol{p}_B \in \mathbb{R}^3$ ($\boldsymbol{q}_B \in \mathbb{H}$) where $\boldsymbol{q}_B$ is the orientation in quaternion and $\mathbb{H}$ is the space of Hamiltonian quaternions, the linear (angular) velocity of O_B in $\mathcal{F}_W$ is denoted as ${}^W\boldsymbol{v}_B \in \mathbb{R}^3$ ($\boldsymbol{\omega}_B \in \mathbb{R}^3$), and finally the linear (angular) acceleration of O_B in $\mathcal{F}_W$ is denoted as ${}^W\dot{\boldsymbol{v}}_B \in \mathbb{R}^3$ ($\dot{\boldsymbol{\omega}}_B \in \mathbb{R}^3$). In addition to the quaternion form, the orientation could be presented as a rotation matrix ${}^W\boldsymbol{R}_B \in SO(3)$ and as Euler angles $(\theta_B, \phi_B, \psi_B)$, where each representation serves a different purpose.

The UAV used in the experiments in this paper is an omnidirectional UAV actuated with eight propellers, where $\boldsymbol{u} \in \mathbb{R}^8$ denote the applied thrust on each propeller. The platform has a weight $m_B \in \mathbb{R}_{>0}$ and and inertia matrix $\boldsymbol{J}_B \in \mathbb{R}^{3\times3}_{>0}$. The propellers apply a wrench on $\mathcal{F}_B$ denoted as $\boldsymbol{w} = [\boldsymbol{f}^\top \in \mathbb{R}^3, \boldsymbol{m}^\top \in \mathbb{R}^3]^\top \in \mathbb{R}^6$. Since the propellers are fixed in $\mathcal{F}_B$, a constant mapping between $\boldsymbol{u}$ and $\boldsymbol{w}$ can be written as $\boldsymbol{w} = \boldsymbol{F}\boldsymbol{u}$, where $\boldsymbol{F} \in \mathbb{R}^{6\times8}$ is the platform's allocation matrix. Accordingly, the platform's Newton-Euler equations of motion can be written as follows:

$$\begin{bmatrix} m_B\dot{\boldsymbol{v}}_B \\ \boldsymbol{J}_B\dot{\boldsymbol{\omega}}_B \end{bmatrix} = \begin{bmatrix} -m_B g\boldsymbol{e}_3 \\ \boldsymbol{\omega}_B \times \boldsymbol{J}_B\boldsymbol{\omega}_B \end{bmatrix} + \begin{bmatrix} {}^W\boldsymbol{R}_B & \boldsymbol{0}_{3\times3} \\ \boldsymbol{0}_{3\times3} & \boldsymbol{I}_3 \end{bmatrix} \boldsymbol{w} + \boldsymbol{d} \quad (1)$$

where $\boldsymbol{d} \in \mathbb{R}^6$ is a disturbance wrench. While disturbances can originate from various sources, we expect the most prominent source in this paper to be the aerodynamic interactions between the airflow generated by the propellers and nearby obstacles.

The UAV is designed such that $\text{rank}(\boldsymbol{F}) = 6$, and as such, it can apply mutually independent wrench. Additionally, it is designed to carry its weight while oriented in any direction.

B. Trajectory Modeling Requirements

The UAV is assumed to be residing in an obstacle-cluttered known environment. Denote by $\boldsymbol{\zeta} = [{}^W\boldsymbol{p}_B{}^\top, \boldsymbol{q}_B^\top]^\top \in \mathcal{C} \subseteq \mathbb{R}^3 \times \mathbb{H}$ as the state of the UAV contained in the configuration space $\mathcal{C}$, by $\dot{\boldsymbol{\zeta}} = [{}^W\boldsymbol{v}_B{}^\top, \boldsymbol{\omega}_B^\top]^\top \in \mathcal{V} \subset \mathbb{R}^6$ as its derivative contained in the velocity space $\mathcal{V}$ and finally by $\ddot{\boldsymbol{\zeta}} = [{}^W\dot{\boldsymbol{v}}_B{}^\top, \dot{\boldsymbol{\omega}}_B^\top]^\top \in \mathcal{A} \subset \mathbb{R}^6$ as its second derivative contained in the acceleration space $\mathcal{A}$. The bounds of $\mathcal{V}$ and $\mathcal{A}$ will be discussed later, where such bounds will be allocated to balance between safety of the computed trajectory and minimum traversal time. The configuration state and its derivatives can be concatenated in the full state $\boldsymbol{\chi} = [\boldsymbol{\zeta}^\top, \dot{\boldsymbol{\zeta}}^\top, \ddot{\boldsymbol{\zeta}}^\top]^\top \in \mathcal{X} \subset \mathbb{R}^3 \times \mathbb{H} \times \mathbb{R}^6 \times \mathbb{R}^6$.

Define the minimal convex hull containing the UAV as $\mathcal{H}(\boldsymbol{\zeta})$, parametrized by the UAV's position and orientation $\boldsymbol{\zeta}$. Additionally, denote an obstacle $\mathcal{O}_b \in \mathbb{R}^3$ as an area in space that cannot be traversed by the UAV, defined with convex and mixed-integer inequalities, defining m half-space constraints:

$$\mathcal{O}_b = \big\{ \boldsymbol{A}_{obs,i}\,{}^W\boldsymbol{p}_B \le \boldsymbol{b}_{obs,i} + \boldsymbol{M}\delta \,| \\ {}^W\boldsymbol{p}_B \in \mathbb{R}^3, \delta = 1(\text{inside openings}) \big\} \quad (2)$$

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 04:05:14 UTC from IEEE Xplore. Restrictions apply.

where $\boldsymbol{A}_{obs,i} \in \mathbb{R}^{m\times 3}$ is a matrix that defines the half-space constraints, $\boldsymbol{b}_{obs,i} \in \mathbb{R}^m$ the corresponding offsets, and the $\boldsymbol{M}\delta$ term allows the incorporation of openings or gaps inside convex obstacles, where $\boldsymbol{M}$ is a matrix with large numbers and $\delta = 1$ only inside the opening. We also define a corresponding obstacle $\mathcal{O}_{b,I}$ inflated by a safety distance d_s, such that $\mathcal{O}_b \subset \mathcal{O}_{b,I}$:

$$\mathcal{O}_{b,I} = \{\boldsymbol{A}_{obs,i}\,{}^W\boldsymbol{p}_B \leq \boldsymbol{b}_{obs,i} + \boldsymbol{M}\delta \pm d_s \mid {}^W\boldsymbol{p}_B \in \mathbb{R}^3, \delta = 1(\text{inside inflated openings})\}. \tag{3}$$

Obstacles and inflated obstacles can be grouped into the set of obstacles $\mathcal{O} = \cup\mathcal{O}_b$ and $\mathcal{O}_I = \cup\mathcal{O}_{b,I}$ respectively.

Following the obstacles definition, it is desired to find a path that avoids collision with obstacles, as such we can define the free $\mathcal{C}_f$ and safe $\mathcal{C}_s$ space as follows:

$$\mathcal{C}_f = \{\boldsymbol{\zeta} \in \mathcal{C} \mid \mathcal{H}(\boldsymbol{\zeta}) \cap \mathcal{O} = \emptyset\}, \tag{4}$$

$$\mathcal{C}_s = \{\boldsymbol{\zeta} \in \mathcal{C} \mid \mathcal{H}(\boldsymbol{\zeta}) \cap \mathcal{O}_I = \emptyset\}. \tag{5}$$

The full state space will then be denoted based on the corresponding configuration space, such as $\mathcal{X}_f = \mathcal{C}_f \times \mathcal{V} \times \mathcal{A}$.

III. Controller Design

The following controller assumes the UAV is following a smooth trajectory, at least twice differentiable $\forall t \in [0, T]$ where T is the time required to follow the path. In what follows any entity $[\cdot]^d$ represents a desired value. To track the desired trajectory $\boldsymbol{\zeta}^d(t)$, the following controller is proposed:

$$\begin{aligned}
{}^W\boldsymbol{f}^d(t) &= gm_B\boldsymbol{z}_W + \dot{\boldsymbol{v}}_B^d + \boldsymbol{K}_p\boldsymbol{e}_p + \boldsymbol{K}_v\boldsymbol{e}_v + \boldsymbol{K}_{i,p}\int_0^t \boldsymbol{e}_p \\
\boldsymbol{m}^d(t) &= \boldsymbol{\omega}_B \times \boldsymbol{J}_B\boldsymbol{\omega}_B + \boldsymbol{J}_B\dot{\boldsymbol{\omega}}_B + \\
&\quad \boldsymbol{K}_R\boldsymbol{e}_R + \boldsymbol{K}_w\boldsymbol{e}_w + \boldsymbol{K}_{i,R}\int_0^t \boldsymbol{e}_R \\
\boldsymbol{w}^d(t) &= \begin{bmatrix} {}^W\boldsymbol{R}_B^\top & \boldsymbol{0}_{3\times3} \\ \boldsymbol{0}_{3\times3} & \boldsymbol{I}_3 \end{bmatrix} \begin{bmatrix} {}^W\boldsymbol{f}^d(t) \\ \boldsymbol{m}^d(t) \end{bmatrix}
\end{aligned} \tag{6}$$

where $\boldsymbol{K}_p, \boldsymbol{K}_v, \boldsymbol{K}_{i,p}, \boldsymbol{K}_R, \boldsymbol{K}_w, \boldsymbol{K}_{i,R} \in \mathbb{R}^{3\times3}$ are matrices with tunable gains along the diagonal of each matrix, while $\boldsymbol{e}_p = {}^W\boldsymbol{p}_B^d(t) - {}^W\boldsymbol{p}_B$, $\boldsymbol{e}_v = {}^W\boldsymbol{v}_B^d(t) - {}^W\boldsymbol{v}_B$, $\boldsymbol{e}_w = \boldsymbol{\omega}_B^d(t) - \boldsymbol{\omega}_B$, and finally, the rotation error is calculated from the rotation matrix, such that $\boldsymbol{e}_R = \frac{1}{2}\left({}^W\boldsymbol{R}_B^{d\,T}(t)\,{}^W\boldsymbol{R}_B - {}^W\boldsymbol{R}_B^T\,{}^W\boldsymbol{R}_B^d(t)\right)^\wedge$, where $[\cdot]^\wedge$ is the inverse skew symmetric operator.

The controller feedback given by (6) linearizes the UAV's dynamical system with respect to the desired wrench $\boldsymbol{w}^d(t)$ [19]. The OmniOcta platform is a fully actuated platform that can apply independent forces and moments in all directions concurrently. As such, $\exists \boldsymbol{u}$ s.t. $\boldsymbol{w}^d(t) = \boldsymbol{F}\boldsymbol{u}\,\forall \boldsymbol{w}^d(t) \mid \|\boldsymbol{w}^d(t)\|_\infty \leq \bar{w}$, for some bound $\bar{w}$ on the desired wrench, and thus $\boldsymbol{w}^d(t)$ allows the asymptotical tracking of the desired trajectory. Since the allocation matrix is invertible, a control input can be found such that $\boldsymbol{u}_c(t) = \boldsymbol{F}^\dagger\boldsymbol{w}^d(t)$, where $\boldsymbol{F}^\dagger$ is the moore-penrose pseudo inverse of $\boldsymbol{F}$. However, as shown in [20], $\boldsymbol{u}_c(t)$ will have at least one negative entry. As such, in this paper the over-actuation of the system is used to ensure a feasible control input that satisfies the desired wrench. Denote by $\boldsymbol{B}$ as the null space basis of $\boldsymbol{F}$, and allocate a corresponding item in the null space of the allocation matrix to satisfy the control limits as follows:

$$\boldsymbol{\lambda}^\star = \arg\min_{\boldsymbol{\lambda}\in\mathbb{R}^2} \|\boldsymbol{u}_c + \boldsymbol{B}\boldsymbol{\lambda}\|, \tag{7}$$

$$\text{s.t. } \boldsymbol{u}_c + \boldsymbol{B}\boldsymbol{\lambda} \geq \boldsymbol{0}, \tag{8}$$

and a corresponding desired control input is chosen such that $\boldsymbol{u}^d(t) = \boldsymbol{u}_c(t) + \boldsymbol{B}\boldsymbol{\lambda}^\star$.

The integral error in Eq. (6) compensates for the unmodeled disturbance $\boldsymbol{d}$. However, an integral term can compensate for constant and slowly varying disturbances. As such, the trajectory planner in this paper will be designed such that the drone slows down near obstacles, where large disturbances are expected, and variance in position could lead to collisions, and then accelerating when traveling in open spaces.

IV. Safe Trajectory Generation

The objective of the trajectory generation is to find a traversable trajectory from a starting configuration $\boldsymbol{\zeta}_s \in \mathcal{C}_f$ to a goal configuration $\boldsymbol{\zeta}_g \in \mathcal{C}_f$. The full problem can be summarized as follows:

Problem 1. Given $\boldsymbol{\zeta}_s$, $\boldsymbol{\zeta}_g$ and $\mathcal{X}_f$, find time T and trajectory $\boldsymbol{\chi}(t)$ such that

$$\begin{aligned}
T = \arg\min_t \boldsymbol{\chi}(t) \in \mathcal{X}_f \\
\text{such that } \boldsymbol{\zeta}(t=0) = \boldsymbol{\zeta}_s, \\
\boldsymbol{\zeta}(t=T) = \boldsymbol{\zeta}_g \\
\dot{\boldsymbol{\zeta}}(0) = \dot{\boldsymbol{\zeta}}(T) = \boldsymbol{0} \\
\ddot{\boldsymbol{\zeta}}(0) = \ddot{\boldsymbol{\zeta}}(T) = \boldsymbol{0}.
\end{aligned} \tag{9}$$

The above problem is untractable for large maps, due to the large state space $\mathcal{X}_f$. Therefore, this paper proposes to solve the above problem by parts, as shown in Algorithm 1, where a path is first computed in the geometrical space $\mathcal{C}_s \subset \mathcal{C}_f$ using the Rapidly Exploring Random Tree algorithm described in Section IV-A, and a trajectory is then fitted to the computed path. The trajectory is computed to balance between slowing down near obstacles to allow for more accurate tracking of the trajectory and accelerating in open spaces, where larger tracking tolerances are acceptable as discussed in Section IV-B and Section IV-C.

A. Rapidly Exploring Random Tree (RRT)

The objective of the path planning algorithm is to find a path within our configuration space $\mathcal{C}_s$. Since our search space is continuous and non-convex, sampling-based motion planning methods were preferred over traditional search-based algorithms like A* and Dijkstra due to their scalability in high-dimensional, continuous, and non-convex spaces. Grid-based approaches struggle with computational efficiency, while sampling-based techniques randomly explore the configuration space, making them more practical for real-world robotics [21].

The Rapidly-exploring Random Tree (RRT) algorithm incrementally expands a tree by sampling random states and linking them to the nearest existing node, ensuring broad coverage of free space. However, RRT does not guarantee optimal paths, leading to improvements such as RRT-Connect and RRT*. RRT-Connect accelerates path finding by growing two trees—one from the start and another from the goal—reducing search space and improving convergence.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 04:05:14 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Trajectory Planning Algorithm

Given: $\mathcal{X}_f$, $\mathcal{O}$, $\boldsymbol{\zeta}_s$ and $\boldsymbol{\zeta}_g$
Compute Safe Space $\mathcal{C}_s$.
Section IV-A: Compute geometric path $\boldsymbol{\zeta}[k] \in \mathcal{C}_s \forall k \in [0, \dots, K]$, such that $\boldsymbol{\zeta}[0] = \boldsymbol{\zeta}_s$ and $\boldsymbol{\zeta}[K] = \boldsymbol{\zeta}_g$.
Eq. (14): Compute $d_{\min,k} \forall k \in [0, \dots, K-1]]$.
Eq. (15): Compute $\overline{v}_k \ \forall k \in [0, \dots, K-1]]$, update $\mathcal{V} \subset \mathcal{X}_f$.
Section IV-C: Fit trajectory to predefined states $\boldsymbol{\zeta}[k] \forall k \in [0, \dots, K]$ while satisfying kinematic constraints from $\mathcal{X}_f$.

Algorithm 2 Distance Computations between Drone to Obstacles in the Direction of Motion

Given: $\mathcal{O}$, $\boldsymbol{\zeta}[k]$, $\mathcal{H}(\boldsymbol{\zeta}[k])$, search radius parameter R_s, maximum radius of drone's convexhull R_d, $\boldsymbol{d}_k$
Compute once for $\mathcal{O}$: compute Axis Aligned Bounding Boxes $A[j]$ for each surface $\boldsymbol{S}[j]$ from $\mathcal{O}_b \subset \mathcal{O}$,
where $j \in 1, \dots, n$ for n surfaces in $\mathcal{O}$.
Find the point $\underline{\boldsymbol{p}}_i$ in $\mathcal{H}(\boldsymbol{\zeta})$ furthest from the direction of motion.
$d_{\min,k} = R_s$.
for $\mathcal{O}_b \in \mathcal{O}$ **do**
  for $j \in \{1, \dots, n\}$ **do**
    Find $\boldsymbol{p}_j^\star = \arg\min_{\boldsymbol{p}_j \in A[j]} \|\underline{\boldsymbol{p}}_i - \boldsymbol{p}_j\|$
    if $\|\underline{\boldsymbol{p}}_i - \boldsymbol{p}_j^\star\| \leq R_s + 2R_d$ and $(\boldsymbol{p}_j^\star - \underline{\boldsymbol{p}}_i) \cdot \boldsymbol{d}_k \geq 0$ **then**
      $d_i = \min_{p_j \in \boldsymbol{S}[j], p_i \in \mathcal{H}(\boldsymbol{\zeta}[k])} \|p_i - p_j\|$.
      $d_{\min,k} = \min(d_{\min,k}, d_i)$.
    end if
  end for
end for

RRT*, on the other hand, refines the paths by selecting parent nodes that minimize cost, resulting in asymptotically optimal solutions [22].

In an aim to find an optimal path in our constrained and high-dimensional environment, in this paper we rely on the RRT* algorithm implementation from the OMPL library [23].

B. Obstacle-Aware Kinematic Constraints

While it is desired to fit the minimum-time trajectory to the computed path, such a path should accommodate for tracking errors when flying close to obstacles. In general, it is expected that the faster the motion (linear and angular) of the UAV, the larger the tracking error.

To account for tracking errors near obstacles, an adaptive speed adjustment strategy is integrated into the kinematic constraints of the trajectory fitting algorithm. This strategy enhances both safety and precision by dynamically adjusting the UAV's velocity. When navigating close to obstacles, the UAV reduces its speed to improve tracking accuracy and control. Conversely, in open spaces, it accelerates to optimize travel time while maintaining safe maneuverability.

Additionally, while the UAV should slow down when approaching an obstacle, its traversal time should not be affected by obstacles in the direction opposite to its direction of motion. For example, after crossing a wall into an open space, the UAV should maintain its maximum allowable speed rather than slowing down due to the presence of the wall behind it. To account for this, at each state $\boldsymbol{\zeta}[k]$, we compute the minimum distance between $\mathcal{H}(\boldsymbol{\zeta}[k])$ and the obstacles in $\mathcal{O}$. However, only obstacles that lie in the half-plane perpendicular to the direction of motion—defined by the vector from $\boldsymbol{\zeta}[k]$ to $\boldsymbol{\zeta}[k+1]$—are considered when evaluating the UAV's speed adjustment.

Denote by $d_{\min}(\mathcal{A}, \mathcal{B})$ as the minimum point-to-point distance between two sets $\mathcal{A}$ and $\mathcal{B}$, *i.e.*, the shortest Euclidean distance between any point in $\mathcal{A}$ and any point in $\mathcal{B}$:

$$d_{\min}(\mathcal{A}, \mathcal{B}) = \min_{\mathbf{p}_a \in \mathcal{A}, \mathbf{p}_b \in \mathcal{B}} \|\mathbf{p}_a - \mathbf{p}_b\| \tag{10}$$

and define the direction of motion at state $\boldsymbol{\zeta}[k]$ as:

$$\boldsymbol{d}_k = \frac{{}^W\boldsymbol{p}_B[k+1] - {}^W\boldsymbol{p}_B[k]}{\|{}^W\boldsymbol{p}_B[k+1] - {}^W\boldsymbol{p}_B[k]\|}. \tag{11}$$

Define the set $\mathcal{O}_k^+$ as the set of obstacles in the positive direction of motion as defined above. The set $\mathcal{O}_k^+$ consists of obstacles located in the positive direction of motion as defined above. $\boldsymbol{p}_i^\star$ denotes the point in $\mathcal{H}(\boldsymbol{\zeta})$ that is farthest from the direction of motion. For motion safety, an obstacle $\mathcal{O}_b$ is considered part of $\mathcal{O}_k^+$ if and only if:

$$(\boldsymbol{p}_j^\star - \boldsymbol{p}_i^\star) \cdot \boldsymbol{d}_k \geq 0, \tag{12}$$

$$s.t.\ \boldsymbol{p}_j^\star = \arg\min_{\boldsymbol{p}_j \in \mathcal{O}_b} \|\boldsymbol{p}_i^\star - \boldsymbol{p}_j\|, \tag{13}$$

In the direction of motion, we define $d_{\min,k}$ as the minimum distance between the UAV and any obstacle at $\boldsymbol{\zeta}[k]$:

$$d_{\min,k} = \min_{\mathcal{O}_b \in \mathcal{O}_k^+} \left\{ \min_{p_j \in \mathcal{O}_b, p_i \in \mathcal{H}(\boldsymbol{\zeta}[k])} \|p_i - p_j\| \right\}. \tag{14}$$

To balance safety with velocity constraints, the velocity limits for the segment $\boldsymbol{s}_k = [\boldsymbol{\zeta}[k], \boldsymbol{\zeta}[k+1]]$ are determined as:

$$\overline{v}_k = \max(\underline{v}, \min(\frac{d_{\min,k+1}}{\overline{d}_{\min}} \overline{v}, \overline{v})) \tag{15}$$

where $\overline{d}_{\min}$ is the threshold distance beyond which the UAV is considered in open space, and $\overline{v}_k$ is computed independently for each axis of $\boldsymbol{\zeta}$. Each axis has distinct velocity constraints with a minimum allowable velocity $\underline{v}$ and a maximum allowable velocity $\overline{v}$, and the final velocity limits are selected in the range $\pm\overline{v}_k$.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 04:05:14 UTC from IEEE Xplore. Restrictions apply.

To efficiently compute $d_{\min,k}$, we restrict the search radius to R_s, a user-specified parameter, and follow Algorithm 2, where R_d is the drone's convexhull maxiumm radius (for a balanced drone it corresponds to the maximum distance between the center of $\mathcal{H}$ and any of its vertices). This method enables rapid computation of $d_{\min,k}$ by filtering out distant obstacles and those positioned opposite to the motion direction before computing the exact distance between $\mathcal{O}_b$ and $\mathcal{H}(\boldsymbol{\zeta}[k])$. The pruning step is applied to axis-aligned bounding boxes to significantly improve computational efficiency, and then accurate distances are computed only for obstacles that were not pruned.

C. Time Optimal Trajectory

After computing the path and determining both the minimum distance between the UAV and any obstacle ($d_{\min,k}$) and the corresponding velocity constraints ($\overline{\boldsymbol{v}}_k$) at each point along the path, we compute the time-optimal trajectory. The `toppra` Python library [24] is employed to generate Bézier curves for $\boldsymbol{\zeta}(t)$, $\dot{\boldsymbol{\zeta}}(t)$, and $\ddot{\boldsymbol{\zeta}}(t)$, ensuring that the trajectory follows all path points in the shortest feasible time while satisfying the given derivative constraints:

$$T = \min \sum_{k=0}^{K-1} \int_{\boldsymbol{s}_k} \frac{ds}{\dot{\boldsymbol{\zeta}}(\boldsymbol{s}_k)} \tag{16}$$

$$\text{s.t.} \quad -\overline{\boldsymbol{v}}_k \leq \dot{\boldsymbol{\zeta}}(\boldsymbol{s}_k) \leq \overline{\boldsymbol{v}}_k \tag{17}$$

$$-\overline{\ddot{\boldsymbol{\zeta}}} \leq \ddot{\boldsymbol{\zeta}} \leq \overline{\ddot{\boldsymbol{\zeta}}} \tag{18}$$

where $\boldsymbol{s}_k$ represents the path segment between points k and $k+1$, and K is the number of segments along the path. The `toppra` library enables variable velocity constraints along each segment, which we utilize to enforce varying velocity limits based on $d_{\min,k}$, while maintaining fixed acceleration limits across all segments. Quaternions and their derivatives are employed throughout the trajectory and path planning process to mitigate singularity issues.

V. Experimental Studies

A. Experimental Setup

The experiments are conducted on the OmniOcta drone presented in [8]. This UAV is a $3.2\,[\mathrm{kg}]$ omnidirectional drone equipped with fixed unidirectional thrusters. It can hover in any orientation and follow independent position and orientation commands. The drone's structure is composed of carbon fiber components, along with custom 3D-printed polyamide and aluminum parts. It has a two-to-one aspect ratio, with a lateral diameter of $1.2\,[\mathrm{m}]$ and a vertical diameter of $0.6\,[\mathrm{m}]$, allowing it to traverse openings vertically that may be impassable horizontally.

The platform is powered by eight Triblade HQ 8040 propellers ($8''$ diameter, 4° pitch), each capable of generating up to 40 N of thrust with the selected ESC-motor combination. A T-motor F7 Pro flight controller is positioned at the UAV's geometric center, interfacing with an onboard PC that computes the desired trajectory and control inputs. The flight controller, running a custom flight software, transmits IMU measurements to the PC and receives desired propeller speeds, which are then relayed to the respective ESCs. The UAV's position and orientation are tracked using a Vicon motion capture system, with markers optimally placed using the algorithm from [25].

Operating in cluttered environments necessitates identifying and mitigating sources of tracking error. In this paper, we identify two primary contributors to tracking errors: the first arising from propeller dynamics (discussed in Section V-B) and the second from aerodynamic interactions with nearby obstacles (discussed in Section V-C).

B. Propeller Dynamics

It is important to highlight the difference in propeller range usage between a conventional quadrotor and the OmniOcta platform. In near-hovering conditions, quadrotors adjust the thrust of each propeller within a relatively narrow range to achieve controlled tilting [26]. Consequently, most quadrotors can implement a sliding controller, ensuring minimal error between the desired and actual thrust [27].

In contrast, the OmniOcta UAV operates in a fundamentally different manner. With propellers positioned in multiple orientations, the platform's weight distribution among the different propellers shifts depending on its hovering orientation. As demonstrated in [8], changes in orientation require some propellers to reduce their thrust to the minimum possible level, while others must rapidly generate significant thrust to support the platform. Additionally, as demonstrated in [28], [29], while omnidirectional platforms are globally fully actuated (all axes are controllable), slow propeller dynamics could render the system only locally controllable. Consequently, when tracking complex and critical trajectories with fully actuated and omnidirectional UAVs, understanding the transient response of each motor-propeller pair is critical for minimizing tracking errors.

Figure 2 illustrates the step response of the propeller during both acceleration and deceleration for different ESC configurations. In all cases, a step input was applied to the motor-propeller system, and the resulting thrust was measured using a force-torque sensor. The figure presents step responses for different Ramp Up Power (RUP) values and braking power levels. The RUP influences the acceleration rate of the propeller, with higher values enabling faster spin-up. Conversely, braking power allows the ESC to apply a braking voltage, thereby controlling the propeller's deceleration rate.

From these plots, the system's normalized transfer functions between desired control input u^d and actual measured one u^a was computed. Each of the transfer functions was identified as a first order transfer function, as:

$$u^a(s) = \frac{1}{\tau s + 1} u^d(s) \tag{19}$$

where τ is the corresponding time constant. During acceleration, the time constant is $0.20\,[\mathrm{s}]$ ($0.15\,[\mathrm{s}]$) for a Ramp Up Power of 50% (150%). During deceleration, the time constant is $0.035\,[\mathrm{s}]$ ($0.21\,[\mathrm{s}]$) when braking power is 50% (0%). It is apparent that relying on the aerodynamic forces to reduce the generated thrust is a rather slow process (requires 3×0.21 sec to reach close to the desired thrust). Hence, despite the additional power dissipation, the braking attribute of each brushless motor is employed.

Given the significant impact of these parameters on motor command tracking, the Ramp Up Power (related to rate limiter) was set to 150% and the braking power to 50%. These settings were chosen to minimize trajectory tracking errors, thereby enabling safer flights when operating in close proximity to obstacles.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 04:05:14 UTC from IEEE Xplore. Restrictions apply.

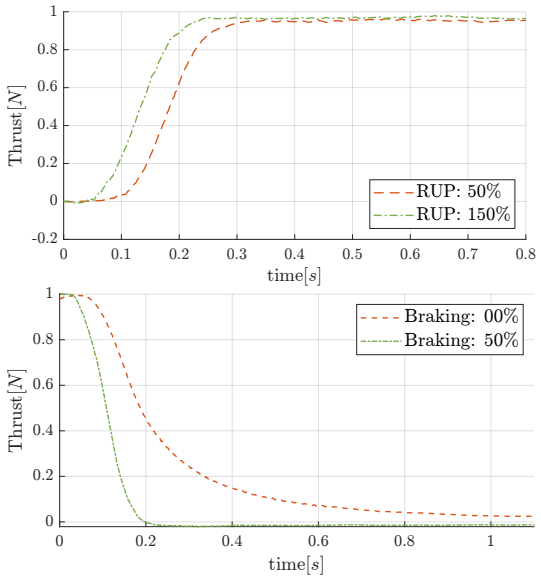


Fig. 2: Step response of the motor-propeller during acceleration and deceleration, for various Ramp Up Power and braking voltages.

C. Aerodynamic Interaction with Obstacles

The tracking error is enlarged when navigating near obstacles due to the unmodeled aerodynamic interactions between the UAV's propellers and nearby surfaces. These interactions can generate a disturbance wrench, resulting in deviations from the intended trajectory. While the modeling and compensation of such disturbances are left for future work, this section experimentally assesses the resulting tracking error.

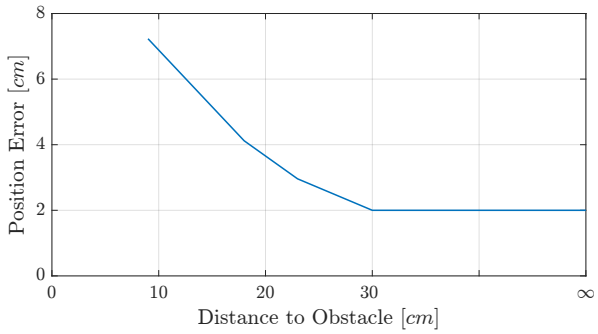


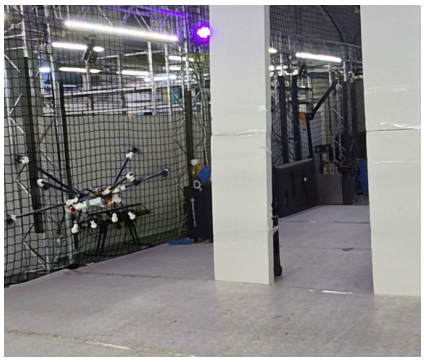
Fig. 3: Maximum position tracking error of the OmniOcta versus separation between the drone and the walls (obstacles).

To evaluate the aerodynamic tracking error near obstacles, the platform was commanded to follow a trajectory passing through the narrow gap shown in Figure 1, with the UAV tilted to a 90° pitch. This was performed while varying the gap size and correspondingly the distance between the UAV and each of the walls.

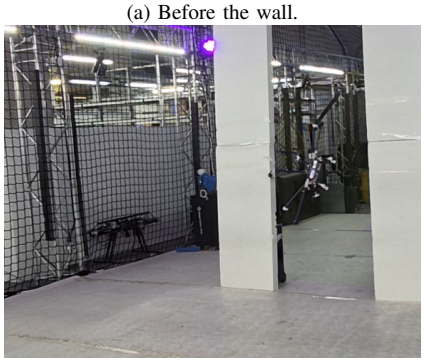
Figure 3 presents the recorded maximum position tracking error as the platform traversed the opening. The results indicate that as the distance between the UAV and the walls of the opening decreases, the tracking error increases. The highest recorded error was 7.5 [cm], observed when the platform was commanded to follow a trajectory with a 9 [cm] clearance from the walls. Further reductions in clearance led to collisions with the walls.

Based on this data, the inflation distance was set to $d_s = 10\,[\mathrm{cm}]$ to account for both expected aerodynamic interactions and potential miscellaneous errors.

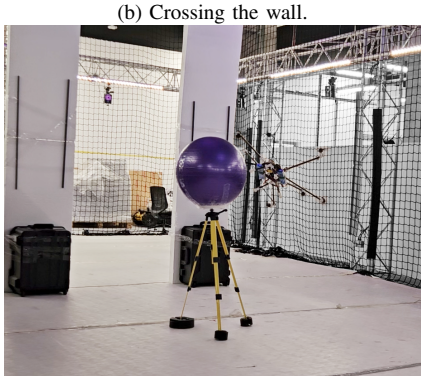
D. Trajectory Tracking in Cluttered Environment



(a) Before the wall.



(b) Crossing the wall.



(c) Going around the sphere.

Fig. 4: Snapshots of the drone along the trajectory while avoiding the obstacles.

In this experiment, the drone is required to track a challenging trajectory in an obstacle cluttered environment. The environment map for the presented experiment has a length of 10 meters and a width of 5 meters. The map has two main obstacles: a narrow vertical opening in a wall at the center of the map and a sphere with a diameter of 53 cm located 2.3 meters from the wall. The drone is placed 3.1 meters from the other side of the wall, and the target is to navigate from the initial position to the other side of the map, and land 4 meters away from the wall, while avoiding the obstacles. The path through the obstacles was calculated offline before the mission started. Additionally, an inflation of $d_s = 10\,[\mathrm{cm}]$ was added to the walls and the sphere to account for tracking errors, as explained in the previous subsection. Figure 4 shows three snapshots of the drone

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 04:05:14 UTC from IEEE Xplore. Restrictions apply.

navigating the physical obstacles at different orientations. The first snapshot captures the drone as it approaches the wall, while the second snapshot shows the drone as it crosses the wall. The final snapshot depicts the drone rotating around the sphere before reaching the end of the trajectory. Similarly, Figure 5 provides a visual illustration of the top map and the desired path.

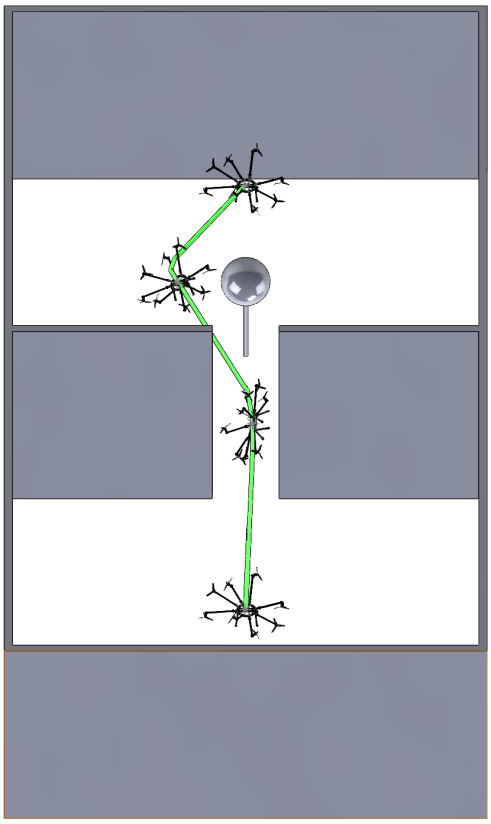


Fig. 5: Obstacle-cluttered environment and drone-trajectory.

After the path was calculated, the trajectory was computed according to Algorithm 1. For each point along the path, the distance between the convex hull of the drone and the closest obstacle ahead, denoted as $d_{\min,k}$, was determined. This distance was used to impose varying linear and angular velocities constraints along the path. The rationale behind employing these varying constraints is to ensure the drone slows down as it approaches obstacles, thereby enabling safe navigation. It is worth mentioning that only obstacles within a radius $R_s = 1\,[\mathrm{m}]$ affect the velocity constraints to prevent excessively conservative behavior. This allows the drone to accelerate between obstacles.

Figure 6 illustrates $d_{\min,k}$ along with the varying allowed and desired linear velocities and quaternion rates throughout the trajectory. The velocity constraints were reduced extensively while crossing the gap to avoid collisions due to tracking errors. Along the trajectory, the quaternion rates consistently reach their maximum allowed values compared to the linear velocities. This highlights the critical importance of maintaining the correct orientation in the test map to avoid collisions with obstacles.

The optimized trajectory was then deployed to the drone and the real pose of the drone while tracking the trajectory was recorded. Figure 7 compares the desired and actual

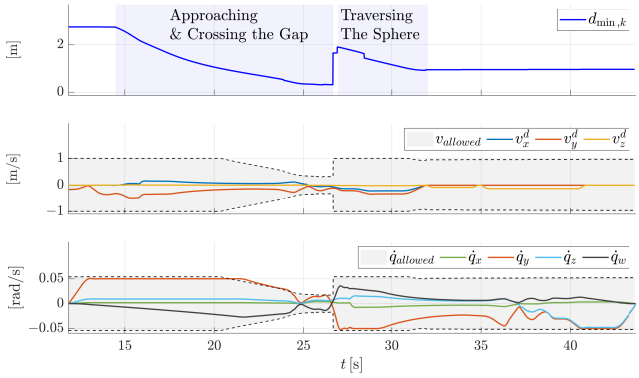


Fig. 6: (Top) $d_{\min,k}$ along the path. (Middle and Bottom) The calculated velocity constraints, along with the resulting desired velocities from the planner.

trajectories. In the bottom figure, the significant change in the pitch angle is clearly noticeable. This change is essential to successfully cross the gap, where the opening cannot be traversed horizontally, while the platform's aspect ratio allows it to pass it vertically.

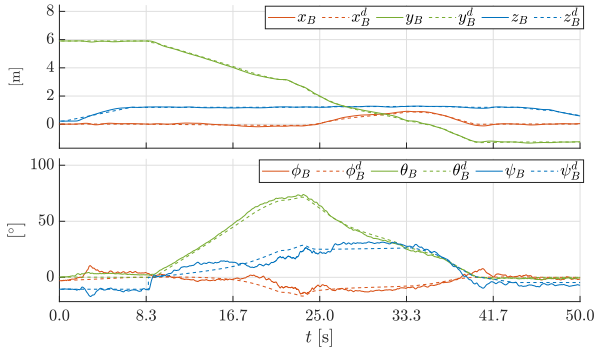


Fig. 7: (Top) Desired versus actual positions along the trajectory. (Bottom) Desired versus actual Euler angles along the trajectory.

Finally, Figure 8 illustrates the desired propeller commands during trajectory tracking. The figure shows that during the transition from the horizontal to the vertical configuration, certain propellers (propellers 2 and 7) had to reduce their thrust to a minimal level, while others that were applying minimal thrust in the horizontal configuration (propellers 4 and 8, overlapping in Figure 8) were required to generate additional thrust to accommodate for the desired wrench at the new configuration.

VI. CONCLUSIONS

This paper presented a trajectory planning framework for omnidirectional drones operating in cluttered environments, ensuring a balance between safety and minimum time traversal. The proposed approach dynamically adjusts speed by decelerating in the presence of obstacles along the direction of motion and accelerating when moving toward open spaces.

The trajectory generation process begins with computing a geometric path from the start to the goal pose. At each point along this path, the minimum distance between the convex hull of the drone and nearby obstacles is determined. Based on this distance, an adaptive velocity constraint is assigned, and a corresponding feasible trajectory is computed passing

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 04:05:14 UTC from IEEE Xplore. Restrictions apply.

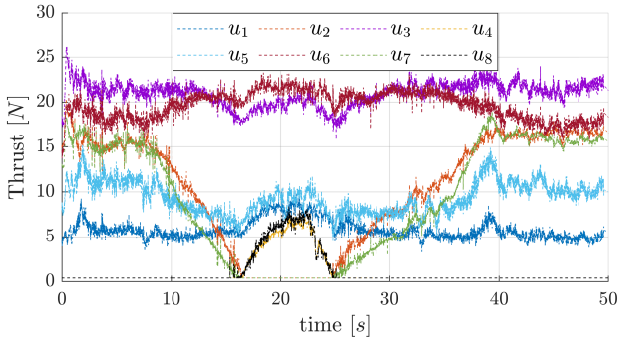


Fig. 8: Desired propeller commands while tracking the trajectory.

through designated waypoints. The effectiveness of the proposed method was validated through real-world experiments, where the drone successfully navigated a cluttered environment with narrow passages. Furthermore, experiments were conducted to identify key sources of trajectory tracking error for the OmniOcta drone, and mitigation strategies were proposed to enhance tracking accuracy.

Future work will focus on integrating online path computation using onboard depth sensors and hybrid strategies that combine path planning with exploration [30], [31], while extending the proposed algorithm to operate safely in the vicinity of dynamic obstacles, such as humans [32]. In addition, we aim to develop a control architecture capable of estimating and compensating for aerodynamic disturbances in real time to enhance tracking performance.

REFERENCES

[1] K. Tong, Y. Hu, B. Dikic, S. Solmaz, F. Fraundorfer, and D. Watzenig, "Robots saving lives: A literature review about search and rescue (sar) in harsh environments," in *2024 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2024, pp. 953–960.

[2] D. Chaikalis, F. Khorrami, and A. Tzes, "Adaptive control approaches for an unmanned aerial manipulation system," in *2020 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2020, pp. 498–503.

[3] K. Ebadi, L. Bernreiter, H. Biggie, G. Catt, Y. Chang, A. Chatterjee, C. E. Denniston, S.-P. Deschênes, K. Harlow, S. Khattak *et al.*, "Present and future of slam in extreme environments: The darpa subt challenge," *IEEE Transactions on Robotics*, vol. 40, pp. 936–959, 2023.

[4] M. Dharmadhikari, N. Khedekar, P. De Petris, M. Kulkarni, M. Nissov, and K. Alexis, "Maritime vessel tank inspection using aerial robots: Experience from the field and dataset release," *arXiv preprint arXiv:2404.19045*, 2024.

[5] A. Ollero, M. Tognon, A. Suarez, D. Lee, and A. Franchi, "Past, Present, and Future of Aerial Robotic Manipulators," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 626–645, 2021.

[6] R. Rashad, J. Goerres, R. Aarts, J. B. Engelen, and S. Stramigioli, "Fully actuated multirotor uavs: A literature review," *IEEE Robotics & Automation Magazine*, vol. 27, no. 3, pp. 97–107, 2020.

[7] M. Hamandi, Q. Sable, M. Tognon, and A. Franchi, "Understanding the Omnidirectional Capability of a Generic Multi-rotor Aerial Vehicle," in *2021 Aerial Robotic Systems Physically Interacting with the Environment (AIRPHARO)*, 2021, pp. 1–6.

[8] M. Hamandi, A. M. Ali, K. Kyriakopoulos, A. Tzes, and F. Khorrami, "An Omnidirectional Non-Tethered Aerial Prototype with Fixed Unidirectional Thrusters," in *2025 International Conference on Robotics and Automation (ICRA)*. Atlanta, GA, USA: IEEE, May 2025, (accepted).

[9] M. Hamandi, A. M. Ali, N. Evangeliou, D. Chaikalis, A. Tzes, K. Kyriakopoulos, and F. Khorrami, "Mechatronic design of an omnidirectional octorotor uav," in *2024 10th International Conference on Automation, Robotics and Applications (ICARA)*, 2024, pp. 300–304.

[10] F. Gao, Y. Lin, and S. Shen, "Gradient-based online safe trajectory generation for quadrotor flight in complex environments," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 3681–3688.

[11] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 2520–2525.

[12] A. Ayyad, M. Chehadeh, P. H. Silva, M. Wahbah, O. A. Hay, I. Boiko, and Y. Zweiri, "Multirotors from Takeoff to Real-time Full Identification using the Modified Relay Feedback Test and Deep Neural Networks," *IEEE Transactions on Control Systems Technology*, vol. 30, no. 4, pp. 1561–1577, 2021.

[13] D. Falanga, E. Mueggler, M. Faessler, and D. Scaramuzza, "Aggressive Quadrotor Flight Through Narrow Gaps with Onboard Sensing and Computing using Active Vision," in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 5774–5781.

[14] D. Brescianini and R. D'Andrea, "Computationally Efficient Trajectory Generation for Fully Actuated Multirotor Vehicles," *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 555–571, 2018.

[15] F. Morbidi, D. Bicego, M. Ryll, and A. Franchi, "Energy-Efficient Trajectory Generation for a Hexarotor with Dual-Tilting Propellers," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 6226–6232.

[16] M. Pantic, L. Ott, C. Cadena, R. Siegwart, and J. Nieto, "Mesh Manifold based Riemannian Motion Planning for Omnidirectional Micro Aerial Vehicles," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4790–4797, 2021.

[17] P. Liu, F. Quan, Y. Liu, and H. Chen, "Collision-free 6-dof Trajectory Generation for Omnidirectional Multi-rotor Aerial Vehicle," *arXiv preprint arXiv:2209.06764*, 2022.

[18] A. M. Ali, M. Hamandi, and A. Tzes, "Efficient safe trajectory planning for an omnidirectional drone," in *2025 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2025, pp. 785–792.

[19] H. K. Khalil and J. W. Grizzle, *Nonlinear systems*. Prentice hall Upper Saddle River, NJ, 2002, vol. 3.

[20] M. Hamandi, K. Sawant, M. Tognon, and A. Franchi, "Omni-plus-seven (o7+): An omnidirectional aerial prototype with a minimal number of unidirectional thrusters," in *2020 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2020, pp. 754–761.

[21] K. Karur, N. Sharma, C. Dharmatti, and J. E. Siegel, "A Survey of Path Planning Algorithms for Mobile Robots," *Vehicles*, vol. 3, no. 3, pp. 448–468, 2021. [Online]. Available: https://www.mdpi.com/2624-8921/3/3/27

[22] S. Karaman and E. Frazzoli, "Sampling-Based Algorithms for Optimal Motion Planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011. [Online]. Available: https://doi.org/10.1177/0278364911406761

[23] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, December 2012, https://ompl.kavrakilab.org.

[24] H. Pham and Q. C. Pham, "A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis," *IEEE Transactions on Robotics*, vol. 34, pp. 645 – 659, 06 2018.

[25] M. Hamandi, A. M. Ali, N. Evangeliou, A. Tzes, and F. Khorrami, "Generating distinctive marker configurations for robot detection in motion capture systems," in *2025 11th International Conference on Automation, Robotics and Applications (ICARA)*. Zagreb, Croatia: IEEE, Feb. 2025, (in print).

[26] N. M. Elocla, M. Chehadeh, I. Boiko, S. Swei, and Y. Zweiri, "The role of time delay in sim2real transfer of reinforcement learning for unmanned aerial vehicles," in *2023 21st International Conference on Advanced Robotics (ICAR)*, 2023, pp. 514–519.

[27] A. Franchi and A. Mallet, "Adaptive closed-loop speed control of bldc motors with applications to multi-rotor aerial vehicles," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 5203–5208.

[28] M. Hamandi, L. Seneviratne, and Y. Zweiri, "Static hovering realization for multirotor aerial vehicles with tiltable propellers," *Journal of Mechanisms and Robotics*, vol. 16, no. 3, p. 031004, 2024.

[29] M. Hamandi, I. Al-Ali, L. Seneviratne, A. Franchi, and Y. Zweiri, "Full-pose trajectory tracking of overactuated multi-rotor aerial vehicles with limited actuation abilities," *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 4951–4958, 2023.

[30] A. Bry and N. Roy, "Rapidly-Exploring Random Belief Trees for Motion Planning Under Uncertainty," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 723–730.

[31] H. U. Unlu, D. Chaikalis, A. Tsoukalas, and A. Tzes, "UAV-Navigation Using Map Slicing and Safe Path Computation," in *2023 9th International Conference on Automation, Robotics and Applications (ICARA)*, 2023, pp. 208–212.

[32] A. Peringal, M. Chehadeh, R. Azzam, M. Hamandi, I. Boiko, and Y. Zweiri, "Design of dynamics invariant lstm for touch based human–uav interaction detection," *IEEE Access*, vol. 10, pp. 116 045–116 058, 2022.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 04:05:14 UTC from IEEE Xplore. Restrictions apply.